

ระบบนัดหมายคลินิกสุขภาพภายในมหาวิทยาลัยและแจ้งเตือนการกินยา

เอกสารวิเคราะห์และออกแบบระบบ (System Analysis & Design Document)

Tech Stack: Frontend/Hosting = Vercel | Backend & Database = Supabase (PostgreSQL + Auth + RLS) ทีมพัฒนา: 6 คน (1 พี่เจอรี่ / 1 คน)

1. User Roles

Role	ชื่อเรียก	สิทธิ์หลัก
Role 1	ผู้ใช้ทั่วไป (Patient) — นักศึกษา/ บุคลากร	สมัคร/ล็อกอิน, ดูบริการคลินิก, ขอนัดหมาย (ไม่ใช่สร้างนัดหมายจริงเอง), ดูสถานะนัดของตัวเอง, ดูรายการยาที่ตัวเองต้องกิน, กดยืนยัน "กินยาแล้ว", ดูประวัติการแจ้งเตือน
Role 2	Admin / เจ้า หน้าที่คลินิก (Staff)	จัดการบัญชีผู้ใช้/บริการคลินิก, อนุมัติ/สร้าง/แก้ไข/ยกเลิกนัดหมายจริงให้ผู้ป่วย, บันทึกการกินยาและตารางกินยาให้ผู้ป่วยแต่ละคน, ดู Dashboard สรุปภาพรวม

หลักการสำคัญ: ผู้ใช้ทั่วไปไม่มีสิทธิ์เขียนตาราง appointments และ medications โดยตรง — ทำได้แค่ "ส่งคำขอ" (appointment_requests) หรืออ่าน/อัปเดตสถานะเฉพาะฟิลด์ที่อนุญาต (เช่น taken_status) ส่วนการสร้าง/อนุมัติ/มอบหมายทำโดย Admin เท่านั้น ซึ่งจะบังคับด้วย Supabase RLS Policy ตาม role ที่เก็บใน profiles.role

2. Database Schema (7 ตาราง)

- profiles — id (FK → auth.users), full_name, student_id, phone, role (patient / admin), created_at
- clinic_services — id, name, description, duration_minutes, is_active, created_by
- appointment_requests — id, patient_id, service_id, preferred_date, preferred_time, reason, status (pending / approved / rejected), created_at
- appointments — id, request_id (nullable), patient_id, staff_id, service_id,

scheduled_date, scheduled_time, status (confirmed / completed / cancelled / no_show), notes, created_by

5. **medications** — id, patient_id, prescribed_by (staff_id), name, dosage, frequency_per_day, start_date, end_date, instructions
6. **medication_reminders** — id, medication_id, reminder_time (time of day), reminder_date, status (pending / taken / missed), taken_at
7. **notifications** — id, user_id, type (appointment / medication / system), title, message, is_read, related_id, created_at

ตารางเสริม (แนะนำเพิ่ม เพื่อให้ระบบสมบูรณ์และป้องกันช่องโหว่จริง)

8. **staff_schedule** — id, staff_id, day_of_week, start_time, end_time, is_available — ใช้เช็คเวลาว่างของเจ้าหน้าที่/แพทย์ก่อนอนุมัตินัด ป้องกันนัดชนกัน (double-booking)
9. **visit_notes** — id, appointment_id (FK), staff_id, diagnosis_summary, recorded_at — บันทึกผลการตรวจสั้นๆ หลังนัดหมายเสร็จสิ้น (status = completed)
10. **audit_logs** — id, actor_id, action (create / update / delete), table_name, record_id, detail (jsonb), created_at — เก็บประวัติการเปลี่ยนแปลงข้อมูลสำคัญ (โดยเฉพาะ appointments/medications) เพื่อ traceability
11. **health_tips** — id, title, content, cover_image_url, created_by, published_at — บทความ/ประกาศสุขภาพที่ Admin โพสต์ให้ผู้ป่วยเห็นในหน้า Dashboard (ตอบโจทย์คำว่า "wellness" ในชื่อโปรเจกต์)
12. **feedback** — id, appointment_id (FK), patient_id, rating (1-5), comment, created_at — ผู้ป่วยให้คะแนนหลังใช้บริการ

ตารางเสริมเหล่านี้ไม่จำเป็นต้องมีสมาชิกดูแลเพิ่ม — แต่ละตารางผูกเข้ากับพีเจอรที่มีอยู่แล้ว (ดูคอลัมน์ "เกี่ยวข้องกับพีเจอร" ในหัวข้อ 8) ทำให้แต่ละคนมีงานเพิ่มเล็กน้อยแต่ทำให้ scope ครบและลึกขึ้น

3. User Journey

3.1 Patient (ผู้ใช้ทั่วไป)

1. เข้าเว็บ → หน้า Landing → กด "เข้าสู่ระบบ / สมัครสมาชิก"
2. Sign up ด้วย Supabase Auth (อีเมล + รหัสผ่าน) → ระบบสร้าง record ใน `profiles` อัตโนมัติ (role = patient)
3. ล็อกอินสำเร็จ → เข้าสู่ **Patient Dashboard** เห็นสรุป: นัดที่จะถึง, ยาที่ต้องกินวันนี้, การแจ้งเตือนล่าสุด

4. ต้องการนัดหมาย → เลือกบริการจาก `clinic_services` → กรอกฟอร์มขอวัน/เวลาที่สะดวก + เหตุผล → ระบบสร้าง record ใน `appointment_requests` (status = pending)
5. รอ Admin อนุมัติ → ได้รับ notification เมื่อสถานะเปลี่ยนเป็น approved (มี record ใหม่ใน `appointments`) หรือ rejected
6. ดูหน้า "นัดหมายของฉัน" → เห็นรายการนัดทั้งหมดพร้อมสถานะ, สามารถขอยกเลิก (ส่งคำขอ ไม่ใช่ลบเอง)
7. ถึงเวลากินยา → ได้รับ notification เตือน (ในเว็บ + อีเมล/push) → เข้าหน้า "ยาของฉันวันนี้" → กดปุ่ม "กินยาแล้ว" → อัปเดต `medication_reminders.status = taken`
8. ดูประวัติการกินยาย้อนหลัง (compliance history) และอ่าน "บทความสุขภาพ" (`health_tips`) ที่ Admin โปสต์ไว้ในหน้า Dashboard
9. หลังนัดหมายเสร็จสิ้น (status = completed) → ได้รับ prompt ให้ให้คะแนน/feedback การใช้บริการ (`feedback`) → Log out

3.2 Admin / เจ้าหน้าที่คลินิก

1. ล็อกอินด้วยบัญชีที่มี role = admin → เข้าสู่ **Admin Dashboard** (กราฟ/สถิติ: จำนวนนัดวันนี้, คำขอค้าง, อัตราการกินยาตรงเวลา)
2. เข้าเมนู "คำขอนัดหมาย" → เห็นรายการ pending requests → กดดูรายละเอียด → ระบบเช็ค `staff_schedule` ว่าช่วงเวลานั้นเจ้าหน้าที่ว่างจริงและไม่ชนกับนัดอื่น → เลือก "อนุมัติ" (สร้าง record `appointments`) หรือ "ปฏิเสธ" พร้อมเหตุผล
3. เข้าเมนู "จัดการนัดหมาย" → ดู/แก้ไข/ยกเลิกนัดที่มีอยู่ (Full CRUD) พร้อม Confirm Dialog ก่อนลบ/ยกเลิก; เมื่อนัดเสร็จสิ้น (completed) สามารถบันทึกผลตรวจสั่งฯ ลง `visit_notes`
4. เข้าเมนู "จัดการยาผู้ป่วย" → เลือกผู้ป่วย → เพิ่มรายการยาใหม่ (ชื่อยา, ขนาด, ความถี่, วันเริ่ม-สิ้นสุด) → ระบบ generate ตาราง `medication_reminders` ล่วงหน้าอัตโนมัติตามความถี่ที่กำหนด
5. เข้าเมนู "จัดการบริการคลินิก" → CRUD บริการ (เพิ่ม/แก้ไข/ปิดใช้งานบริการ)
6. เข้าเมนู "บทความสุขภาพ" → CRUD โปสต์ `health_tips` ให้ผู้ป่วยเห็นในหน้า Dashboard
7. ดูรายงานสรุป (การมาตามนัด, no-show rate, compliance การกินยา, คะแนน feedback เฉลี่ย) → กด Export เป็น CSV/PDF → Log out

4. Features Specification (6 ฟีเจอร์ / 6 สมาชิก)

#	ฟีเจอร์	ผู้รับผิดชอบ	ตารางหลักที่เกี่ยวข้อง	ตารางเสริมที่ผูกด้วย	CRUD ที่ครอบคลุม

1	Authentication & Profile Management	สมาชิก 1	profiles	—	Create (สมัคร), Read (ดู โปรไฟล์), Update (แก้ไข ข้อมูลส่วนตัว), Delete (ปิด บัญชี/Admin ลบ user)
2	Appointment Request (ฝั่งผู้ป่วย)	สมาชิก 2	appointment_requests , clinic_services	feedback	Create (ส่ง คำขอ/ให้คะแนน หลังใช้บริการ), Read (ดู สถานะ), Update (แก้ไข คำขอก่อน อนุมัติ), Delete (ยกเลิกคำขอ)
3	Appointment Management (ฝั่ง Admin)	สมาชิก 3	appointments , appointment_requests	staff_schedule , visit_notes	Create (อนุมัติ/ สร้างนัดจริง/เช็ค ตารางว่าง), Read (ดูตาราง นัดทั้งหมด), Update (แก้ไข/ เลื่อนนัด/บันทึก ผลตรวจ), Delete (ยกเลิก นัด)
4	Medication Management (ฝั่ง Admin)	สมาชิก 4	medications , medication_reminders	—	Create (สั่ง ยา+สร้างตาราง เตือน), Read (ดู รายการยาผู้ ป่วย), Update (แก้ไขขนาด/ ความถี่), Delete (หยุด ยา)
	Medication Reminder &	สมาชิก			Create (ระบบ auto- generate), Read (ดู รายการวันนี้/

5	Tracking (ฝั่งผู้ป่วย)	5	medication_reminders	—	ประวัติ), Update (mark taken/missed), Delete (Admin ลบ reminder ผิดพลาด)
6	Notification Center & Admin Dashboard	สมาชิก 6	notifications , ทุกตาราง (aggregate)	audit_logs , health_tips	Create (ระบบสร้าง noti อัตโนมัติ/โพสต์บทความ), Read (ดู noti/สถิติ/audit log), Update (mark as read/แก้ไขบทความ), Delete (ลบ noti เก่า/ลบบทความ)

ทุกฟีเจอร์ต้องมี **Authentication Guard** (เช็ค session + role ก่อนเข้าถึงหน้า/API) และ **Supabase RLS Policy** เฉพาะของตาราง

5. User Stories

Feature 1: Authentication & Profile Management

- **US1.1 (F):** ในฐานะผู้ใช้ใหม่, ฉันต้องการสมัครสมาชิกด้วยอีเมลและรหัสผ่าน, เพื่อที่จะสามารถเข้าใช้งานระบบได้
- **US1.2 (F):** ในฐานะผู้ใช้, ฉันต้องการแก้ไขข้อมูลส่วนตัว (ชื่อ, เบอร์โทร, รหัสนักศึกษา), เพื่อที่จะให้ข้อมูลติดต่อเป็นปัจจุบัน
- **US1.3 (F):** ในฐานะ Admin, ฉันต้องการดูและจัดการรายชื่อผู้ใช้ทั้งหมด, เพื่อที่จะควบคุมสิทธิ์การเข้าถึงระบบ
- **US1.4 (NF):** ในฐานะระบบ, ฉันต้องการเข้ารหัสรหัสผ่านและจำกัดการเข้าถึงข้อมูลตาม role, เพื่อที่จะรักษาความปลอดภัยของข้อมูลผู้ใช้

Feature 2: Appointment Request

- **US2.1 (F):** ในฐานะผู้ป่วย, ฉันต้องการเลือกบริการและส่งคำขอวันเวลาที่สะดวก, เพื่อที่จะขอเข้ารับบริการที่คลินิก

- **US2.2 (F):** ในฐานะผู้ป่วย, ฉันต้องการดูสถานะคำขออนุญาตหมายของตัวเอง, เพื่อที่จะทราบว่าคำขอถูกอนุมัติหรือไม่
- **US2.3 (F):** ในฐานะผู้ป่วย, ฉันต้องการยกเลิกคำขอที่ยังไม่ได้รับการอนุมัติ, เพื่อที่จะเปลี่ยนแปลงแผนการนัดได้
- **US2.4 (NF):** ในฐานะระบบ, ฉันต้องการแสดง Loading state ขณะดึงข้อมูลบริการและป้องกันการส่งคำขอซ้ำ, เพื่อที่จะให้ประสบการณ์ใช้งานราบรื่นและข้อมูลถูกต้อง
- **US2.5 (F):** ในฐานะผู้ป่วย, ฉันต้องการให้คะแนนและแสดงความคิดเห็นหลังใช้บริการเสร็จสิ้น, เพื่อที่จะช่วยให้คลินิกพัฒนาคุณภาพบริการ

Feature 3: Appointment Management (Admin)

- **US3.1 (F):** ในฐานะ Admin, ฉันต้องการดูรายการคำขออนุญาตหมายที่รอดำเนินการ, เพื่อที่จะพิจารณาอนุมัติหรือปฏิเสธ
- **US3.2 (F):** ในฐานะ Admin, ฉันต้องการอนุมัติคำขอและกำหนดวันเวลา/เจ้าหน้าที่จริง, เพื่อที่จะสร้างนัดหมายที่ยืนยันแล้ว
- **US3.3 (F):** ในฐานะ Admin, ฉันต้องการแก้ไขหรือยกเลิกนัดหมายที่มีอยู่, เพื่อที่จะจัดการตารางนัดให้ทันสถานการณ์จริง
- **US3.4 (NF):** ในฐานะ Admin, ฉันต้องการเห็น Confirm Dialog ก่อนยกเลิกนัด, เพื่อที่จะป้องกันการยกเลิกโดยไม่ตั้งใจ
- **US3.5 (NF):** ในฐานะระบบ, ฉันต้องการตรวจสอบตารางเวลาว่างของเจ้าหน้าที่ก่อนอนุมัตินัด, เพื่อที่จะป้องกันการนัดหมายชนกัน (double-booking)
- **US3.6 (F):** ในฐานะ Admin, ฉันต้องการบันทึกผลการตรวจสั้นๆ หลังนัดหมายเสร็จสิ้น, เพื่อที่จะเก็บประวัติการรักษาของผู้ป่วยไว้อ้างอิง

Feature 4: Medication Management (Admin)

- **US4.1 (F):** ในฐานะ Admin, ฉันต้องการบันทึกการยาให้ผู้ป่วยแต่ละคน (ชื่อยา, ขนาด, ความถี่, ระยะเวลา), เพื่อที่จะกำหนดแผนการรักษา
- **US4.2 (F):** ในฐานะ Admin, ฉันต้องการแก้ไขหรือหยุดรายการยาที่บันทึกไว้, เพื่อที่จะปรับแผนการรักษาตามอาการผู้ป่วย
- **US4.3 (F):** ในฐานะ Admin, ฉันต้องการให้ระบบสร้างตารางแจ้งเตือนอัตโนมัติเมื่อบันทึกยาใหม่, เพื่อที่จะไม่ต้องสร้างการแจ้งเตือนทีละรายการเอง
- **US4.4 (NF):** ในฐานะระบบ, ฉันต้องการตรวจสอบความถูกต้องของข้อมูล (เช่น วันสิ้นสุด > วันเริ่มต้น) ก่อนบันทึก, เพื่อที่จะป้องกันข้อมูลผิดพลาด

Feature 5: Medication Reminder & Tracking

- **US5.1 (F):** ในฐานะผู้ป่วย, ฉันต้องการดูรายการยาที่ต้องกินในวันนี้, เพื่อที่จะกินยาได้ตรงเวลา
- **US5.2 (F):** ในฐานะผู้ป่วย, ฉันต้องการกดยืนยัน "กินยาแล้ว" ในแต่ละมือ, เพื่อที่จะบันทึกการกินยาของตัวเอง
- **US5.3 (F):** ในฐานะผู้ป่วย, ฉันต้องการดูประวัติการกินยาย้อนหลัง, เพื่อที่จะติดตามความสม่ำเสมอของตัวเอง
- **US5.4 (NF):** ในฐานะระบบ, ฉันต้องการอัปเดตสถานะเป็น "missed" อัตโนมัติหากเลยเวลาที่กำหนดโดยไม่มีการยืนยัน, เพื่อที่จะให้ข้อมูล compliance แม่นยำ
- **US5.5 (NF):** ในฐานะผู้ป่วย, ฉันต้องการได้รับการแจ้งเตือนกินยาผ่านอีเมลหรือ push notification นอกเหนือจากใน-แอป, เพื่อที่จะไม่พลาดเวลากินยาแม้ไม่ได้เปิดเว็บอยู่

Feature 6: Notification Center & Admin Dashboard

- **US6.1 (F):** ในฐานะผู้ใช้ (ทุก role), ฉันต้องการเห็นรายการแจ้งเตือนทั้งหมดของตัวเอง, เพื่อที่จะไม่พลาดข้อมูลสำคัญ
- **US6.2 (F):** ในฐานะผู้ใช้, ฉันต้องการทำเครื่องหมายว่าอ่านแจ้งเตือนแล้ว, เพื่อที่จะจัดการรายการแจ้งเตือนของตัวเอง
- **US6.3 (F):** ในฐานะ Admin, ฉันต้องการดู Dashboard สรุปสถิติ (จำนวนนัด, compliance การกินยา, no-show rate), เพื่อที่จะประเมินภาพรวมการดำเนินงานของคลินิก
- **US6.4 (NF):** ในฐานะระบบ, ฉันต้องการจำกัดไม่ให้ผู้ใช้เห็นแจ้งเตือนของผู้อื่น, เพื่อที่จะรักษาความเป็นส่วนตัวของข้อมูลสุขภาพ
- **US6.5 (NF):** ในฐานะ Admin, ฉันต้องการดูประวัติการเปลี่ยนแปลงข้อมูลสำคัญ (audit log), เพื่อที่จะตรวจสอบย้อนหลังได้หากเกิดข้อผิดพลาดหรือข้อพิพาท
- **US6.6 (F):** ในฐานะ Admin, ฉันต้องการโพสต์บทความ/ประกาศสุขภาพและ export รายงานเป็นไฟล์, เพื่อที่จะสื่อสารกับผู้ใช้และนำข้อมูลไปใช้ต่อได้

6. Acceptance Criteria (ละเอียดเชิง Technical & UI)

Feature 1: Authentication & Profile Management

ID	User Story	Acceptance Criteria
AC1.1	US1.1	มีฟอร์ม Sign Up (email, password, confirm password) เรียกใช้ <code>supabase.auth.signUp()</code> ; แสดง validation error แบบ inline (เช่น email format, password ≥ 8 ตัวอักษร); เมื่อสำเร็จให้ trigger/Edge Function สร้าง row ใน <code>profiles</code> โดยอัตโนมัติด้วย role default = <code>patient</code> ; แสดง Loading spinner

		ขณะส่งข้อมูล และ Toast/Alert แจ้งผลสำเร็จหรือ error
AC1.2	US1.2	มีหน้า "โปรไฟล์ของฉัน" ดึงข้อมูลจริงจาก profiles ด้วย <code>select().eq('id', user.id)</code> ; ฟอรั่มแก้ไขมีปุ่ม Save ที่เรียก <code>update()</code> เฉพาะ row ของตนเอง; ต้องมี RLS Policy <code>UPDATE</code> บนตาราง profiles ที่อนุญาตเฉพาะ <code>auth.uid() = id</code> ; แสดง Loading state ขณะโหลด/บันทึก และ Error state หากบันทึกล้มเหลว
AC1.3	US1.3	หน้า Admin "จัดการผู้ใช้" แสดงตาราง (DataTable) ดึงข้อมูลทุก row จาก profiles (ต้องมี RLS <code>SELECT</code> ที่อนุญาตเฉพาะ <code>role = admin</code> เห็นทุกคน); มีปุ่ม Delete/Deactivate พร้อม Confirm Dialog ระบุชื่อผู้ใช้ที่จะลบ; มี Search/Filter ตามชื่อหรือ role
AC1.4	US1.4	ตั้งค่า RLS แยกตาม role โดยใช้ helper function <code>is_admin()</code> หรือเช็ค <code>profiles.role</code> ใน policy ของทุกตาราง; หน้า Protected Route ทุกหน้าต้องเช็ค <code>supabase.auth.getSession()</code> ก่อน render และ redirect ไปหน้า login หากไม่มี session

Feature 2: Appointment Request (ผู้ป่วย)

ID	User Story	Acceptance Criteria
AC2.1	US2.1	มี Dropdown ดึงรายการบริการจริงจาก <code>clinic_services</code> (where <code>is_active = true</code>); ฟอรั่มมีช่องเลือกวัน (date picker, ไม่ให้เลือกวันในอดีต), เวลาที่สะดวก, และช่อง reason (textarea, required); กดปุ่ม "ส่งคำขอ" เรียก <code>insert()</code> ลง <code>appointment_requests</code> พร้อม <code>patient_id = auth.uid()</code> ; แสดง Loading ระหว่าง submit และ Success message เมื่อสำเร็จ
AC2.2	US2.2	หน้า "คำขอของฉัน" ดึงข้อมูลด้วย <code>select().eq('patient_id', auth.uid())</code> ; แสดง Badge สีตามสถานะ (เหลือง=pending, เขียว=approved, แดง=rejected); ต้องมี RLS <code>SELECT</code> ที่จำกัดให้เห็นเฉพาะคำขอของตัวเอง
AC2.3	US2.3	ปุ่ม "ยกเลิกคำขอ" ปรากฏเฉพาะเมื่อ <code>status = pending</code> ; กดแล้วแสดง Confirm Dialog ; ยืนยันแล้วเรียก <code>update()</code> เปลี่ยน status เป็น <code>cancelled</code> (ไม่ใช่ hard delete เพื่อเก็บ audit log); RLS <code>UPDATE</code> อนุญาตเฉพาะเจ้าของคำขอและเฉพาะ <code>status = pending</code> เท่านั้น
AC2.4	US2.4	ปุ่ม Submit ถูก disable ระหว่างรอ response (ป้องกันกดซ้ำ); แสดง Skeleton loader ขณะโหลดรายการบริการ; แสดงข้อความ Error พร้อมปุ่ม Retry หากดึงข้อมูลจาก Supabase ล้มเหลว
AC2.5	US2.5	เมื่อ <code>appointments.status = 'completed'</code> ระบบแสดง Modal/Banner ให้ผู้ป่วยกดให้คะแนน (star rating 1-5) พร้อม comment (textarea, optional); กด Submit เรียก <code>insert()</code> ลง <code>feedback</code> ; แสดงเฉพาะครั้งเดียวต่อนัดหมาย (เช็คว่ามี feedback อยู่แล้วหรือยังก่อนแสดง); RLS <code>INSERT</code> อนุญาตเฉพาะเจ้าของ

		appointment
--	--	-------------

Feature 3: Appointment Management (Admin)

ID	User Story	Acceptance Criteria
AC3.1	US3.1	หน้า Admin แสดงตารางคำขอ where status = 'pending' , เรียงตามวันที่ขอ (ascending); แต่ละแถวมีปุ่ม "ดูรายละเอียด" เปิด Modal แสดงข้อมูลผู้ป่วยและ reason แบบเต็ม; ต้องมี RLS SELECT บน appointment_requests ที่อนุญาต เฉพาะ role admin เห็นทุก request
AC3.2	US3.2	ใน Modal มีฟอร์มให้ Admin กำหนด scheduled_date, scheduled_time, staff_id (dropdown จาก profiles ที่ role=admin); กด "อนุมัติ" เรียก transaction: insert() ลง appointments และ update() status ของ request เป็น approved ; กด "ปฏิเสธ" ให้กรอกเหตุผล แล้ว update() status เป็น rejected ; ทั้งสองปุ่มต้องมี Confirm Dialog
AC3.3	US3.3	หน้า "ตารางนัดหมายทั้งหมด" แสดงเป็น Table/Calendar view ดึงจาก appointments จริง; มีปุ่ม Edit เปิดฟอร์มแก้ไขวัน/เวลา/สถานะ (เรียก update()); ปุ่ม Delete/Cancel มี Confirm Dialog ระบุชื่อผู้ป่วยและวันที่นัด ก่อนเรียก update() เปลี่ยน status เป็น cancelled
AC3.4	US3.4	Confirm Dialog ต้องแสดงข้อความเจาะจง เช่น "ยืนยันยกเลิกนัดหมายของ [ชื่อผู้ป่วย] วันที่ [วันที่]?" พร้อมปุ่ม ยืนยัน/ยกเลิก; หลังยืนยันสำเร็จต้องมี Toast แจ้งผลและ trigger การสร้าง notification ไปยังผู้ป่วยที่เกี่ยวข้อง
AC3.5	US3.5	ก่อนบันทึกนัดหมายใหม่ ระบบต้อง query appointments ที่มี staff_id และช่วง เวลาซ้อนทับกับเวลาที่เลือก (scheduled_date + scheduled_time ± duration) และเช็คว่า staff_schedule ว่าอยู่ในช่วงเวลาวางจริง; หากชนกันให้แสดง Error message ระบุเวลาที่ไม่ว่าง และ block การกด "อนุมัติ" จนกว่าจะเปลี่ยนเวลา
AC3.6	US3.6	ในหน้ารายละเอียดนัดที่ status = completed มีฟอร์มบันทึกผลตรวจ (diagnosis_summary, textarea) เรียก insert() / upsert() ลง visit_notes ผูกกับ appointment_id ; RLS SELECT / INSERT บน visit_notes อนุญาตเฉพาะ role admin เขียน แต่ผู้ป่วยเจ้าของนัดอ่านได้อย่างเดียว (read-only)

Feature 4: Medication Management (Admin)

ID	User Story	Acceptance Criteria
		ฟอร์มเพิ่มยาใหม่ ประกอบด้วย: dropdown เลือกผู้ป่วย (ดึงจาก profiles where

AC4.1	US4.1	role=patient), ชื่อยา (text), ขนาด/dosage (text), ความถี่ต่อวัน (number input), วันเริ่ม-สิ้นสุด (date range picker), instructions (textarea); validation ต้อง block หาก <code>end_date < start_date</code> ; กด Save เรียก <code>insert()</code> ลง <code>medications</code>
AC4.2	US4.2	หน้า “รายการยาผู้ป่วย” มีปุ่ม Edit ต่อแถว เปิดฟอร์มแก้ไข (pre-filled ด้วยข้อมูลจริงจาก DB); ปุ่ม “หยุดยา” มี Confirm Dialog แล้ว <code>update()</code> <code>end_date</code> เป็นวันปัจจุบัน (soft-stop ไม่ใช่ hard delete); RLS <code>UPDATE / DELETE</code> บน <code>medications</code> อนุญาตเฉพาะ role admin
AC4.3	US4.3	เมื่อบันทึก <code>medications</code> สำเร็จ ระบบต้องเรียก Function/Trigger เพื่อ generate rows ใน <code>medication_reminders</code> ล่วงหน้าตาม frequency และช่วงวันที่ (เช่น ยา 2 มื้อ/วัน x 7 วัน = 14 rows); แสดง Loading indicator ระหว่าง generate และ Success message สรุปจำนวน reminder ที่สร้าง
AC4.4	US4.4	Client-side validation แสดง error message ได้ input ทันทีเมื่อกรอกผิด (เช่น “วันสิ้นสุดต้องมากกว่าวันเริ่มต้น”); Server-side ต้องมี CHECK constraint ในตาราง <code>medications (end_date >= start_date)</code> เป็นชั้นป้องกันที่สอง
AC4.5	US4.1	หน้า “รายการยาทั้งหมด” มี Search box (ค้นหาตามชื่อผู้ป่วย/ชื่อยา) และ Pagination (ดึงข้อมูลที่ละหน้า ~20 rows ด้วย <code>.range()</code>) แทนการดึงข้อมูลทั้งหมดมาแสดงครั้งเดียว เพื่อรองรับข้อมูลจำนวนมาก

Feature 5: Medication Reminder & Tracking

ID	User Story	Acceptance Criteria
AC5.1	US5.1	หน้า “ยาวันนี้” ดึงข้อมูลจริงจาก <code>medication_reminders</code> join <code>medications</code> where <code>patient_id = auth.uid()</code> และ <code>reminder_date = today</code> ; แสดงเป็น list เรียงตามเวลา พร้อม icon สถานะ (pending/taken/missed); RLS <code>SELECT</code> จำกัดเฉพาะ reminder ของตัวเอง
AC5.2	US5.2	แต่ละรายการมีปุ่ม “กินยาแล้ว” เรียก <code>update()</code> เปลี่ยน status เป็น taken และบันทึก <code>taken_at = now()</code> ; ปุ่ม disable ทันทีหลังกด (optimistic UI) พร้อม Loading spinner เล็กจนกว่า request จะสำเร็จ; RLS <code>UPDATE</code> อนุญาตเฉพาะเจ้าของ reminder และจำกัดให้แก้ไขได้เฉพาะ field status/taken_at เท่านั้น
AC5.3	US5.3	หน้า “ประวัติการกินยา” มี date range filter ดึงข้อมูลจาก <code>medication_reminders</code> ย้อนหลัง; แสดงเป็นตารางหรือ Calendar heatmap พร้อมสรุป % compliance (taken/total); มี Empty state หากไม่มีข้อมูลในช่วงที่เลือก
		มี Scheduled Function (Supabase Cron/Edge Function) รันทุกชั่วโมงเพื่อ <code>update()</code> reminder ที่ <code>reminder_time</code> ผ่านไปแล้ว <code>> threshold</code> ที่กำหนด (เช่น

AC5.4	US5.4	2 ชม.) และยัง status = pending ให้เปลี่ยนเป็น missed ; ต้องมี log การรันเพื่อ debug
AC5.5	US5.5	มี Edge Function ที่ Supabase Cron เรียกใกล้ถึง reminder_time (เช่น ก่อน 15 นาที) เพื่อส่งอีเมลผ่าน Resend/SendGrid API ไปยังอีเมลผู้ป่วย หรือ Web Push Notification (ถ้าผู้ใช้อนุญาต browser permission); เก็บสถานะการส่งไว้ใน notifications เพื่อป้องกันส่งซ้ำ

Feature 6: Notification Center & Admin Dashboard

ID	User Story	Acceptance Criteria
AC6.1	US6.1	Icon กระดิ่งที่ Navbar แสดง badge จำนวน unread notification (ดึงจาก notifications where user_id = auth.uid() and is_read = false); คลิกเปิด Dropdown/หน้ารายการแบบ real-time (ใช้ Supabase Realtime subscription) หรือ polling; RLS SELECT จำกัดเฉพาะ noti ของตนเอง
AC6.2	US6.2	แต่ละ notification มีปุ่ม/การคลิกเพื่อ mark as read เรียก update() เปลี่ยน is_read = true ; มีปุ่ม “อ่านทั้งหมด” (mark all as read); มีปุ่มลบ notification พร้อม Confirm Dialog สำหรับ noti เก่า
AC6.3	US6.3	หน้า Admin Dashboard ดึงข้อมูลจริงด้วย aggregate query (COUNT, AVG) จาก appointments และ medication_reminders ; แสดงเป็น Chart (เช่น bar chart จำนวนนัด/วัน, pie chart อัตรา taken/missed) พร้อม Loading skeleton ขณะคำนวณ; ต้องมี RLS ที่อนุญาตเฉพาะ role admin query ข้อมูลรวมของทุกคนได้
AC6.4	US6.4	RLS Policy บนตาราง notifications ต้องกำหนด SELECT , UPDATE , DELETE โดยเช็ค auth.uid() = user_id เท่านั้น (ยกเว้น admin ที่มี policy แยกสำหรับดู system-level notification); มีการทดสอบ (manual test case) ยืนยันว่า user A ไม่สามารถดู/แก้ไข noti ของ user B ได้แม้จะเดา id
AC6.5	US6.5	มี Database Trigger บนตาราง appointments และ medications ที่ insert record ลง audit_logs ทุกครั้งที่มีการ create/update/delete (เก็บ actor_id, action, ค่าก่อน-หลังใน detail แบบ jsonb); หน้า Admin “ประวัติการเปลี่ยนแปลง” แสดงตารางพร้อม filter ตามวันที่/ผู้กระทำ; RLS SELECT บน audit_logs อนุญาตเฉพาะ role admin
AC6.6	US6.6	หน้า “บทความสุขภาพ” มีฟอร์ม Create/Edit (title, content, cover image upload ผ่าน Supabase Storage) และปุ่ม Delete พร้อม Confirm Dialog เรียก CRUD บน health_tips ; หน้า Dashboard รายงานมีปุ่ม “Export CSV/PDF” ที่ generate ไฟล์จากข้อมูล aggregate จริง (ไม่ใช่ mock data) ให้ดาวน์โหลดทันที

7. Non-Functional Requirements ระดับระบบ (กลุ่มทุกพีเจอร်)

หัวข้อ	รายละเอียด
Responsive / Mobile-first	ทุกหน้าต้องแสดงผลถูกต้องบนหน้าจอมือถือ (ผู้ใช้หลักคือนักศึกษาที่เปิดผ่านมือถือ) — ใช้ Tailwind breakpoints ทดสอบที่ 375px, 768px, 1024px เป็นอย่างน้อย
Data Privacy (ข้อมูลสุขภาพ)	ห้าม log ข้อมูลผู้ป่วย/ยา/ผลตรวจลง console หรือ error tracking แบบ plaintext; จำกัดสิทธิ์เห็นข้อมูลสุขภาพเฉพาะเจ้าของและ Admin เท่านั้นผ่าน RLS ทุกตารางที่เกี่ยวข้อง; มีนโยบายลบ/anonymize ข้อมูลผู้ป่วยที่จบการศึกษาแล้ว
Performance	ทุกหน้ารายการ (นัดหมาย, ยา, ผู้ใช้) ต้องมี pagination หรือ infinite scroll ไม่ query ข้อมูลทั้งหมดมาแสดงในครั้งเดียวหากเกิน ~50 rows
Error Handling มาตรฐาน	ทุกหน้าที่เรียก Supabase ต้องมี 3 state ครบ: Loading (skeleton/spinner), Empty (ไม่มีข้อมูล), Error (ข้อความ + ปุ่ม Retry)
Security	ทุก Policy ต้องผ่านการทดสอบ manual (เช่น พยายามเข้าถึงข้อมูลของ user อื่นด้วย id ที่เดาได้) ก่อน submit งาน

8. หมายเหตุสำหรับการต่อยอด

- **Use Case Diagram:** ใช้ 2 Actor (Patient, Admin) ตาม User Journey ในหัวข้อ 3 — แต่ละ use case สอดคล้องกับ User Story ในหัวข้อ 5 โดยตรง (เช่น “ส่งคำขอนัดหมาย”, “อนุมัตินัดหมาย”, “บันทึกการยา”, “ยืนยันกินยา”, “ให้คะแนนบริการ”, “โพสต์บทความสุขภาพ”)
- **ER Diagram:** ใช้ 12 ตารางในหัวข้อ 2 เป็นฐาน (7 ตารางหลัก + 5 ตารางเสริม) โดย `medication_reminders / appointments / visit_notes / feedback` มี Foreign Key ย้อนกลับไปยัง `medications / appointment_requests / appointments` ตามลำดับ เพื่อรักษา referential integrity
- **RLS Strategy โดยรวม:** แนะนำสร้าง SQL function `is_admin()` ที่เช็ค `profiles.role = 'admin'` แล้วเรียกใช้ซ้ำในทุก Policy เพื่อลดความซ้ำซ้อนของโค้ด SQL
- **ลำดับความสำคัญหากเวลาไม่พอ:** ตารางเสริม (`staff_schedule`, `visit_notes`, `audit_logs`, `health_tips`, `feedback`) ออกแบบให้เป็น “ส่วนเสริม” ที่แต่ละคนเพิ่มเข้าไปในพีเจอร်ตัวเองได้ที่หลัง โดยไม่กระทบ 7 ตารางหลักและ Use Case/ER Diagram หลักที่ทำไว้แล้ว หากเวลาไม่พอสามารถตัดออกได้โดยไม่กระทบ core flow